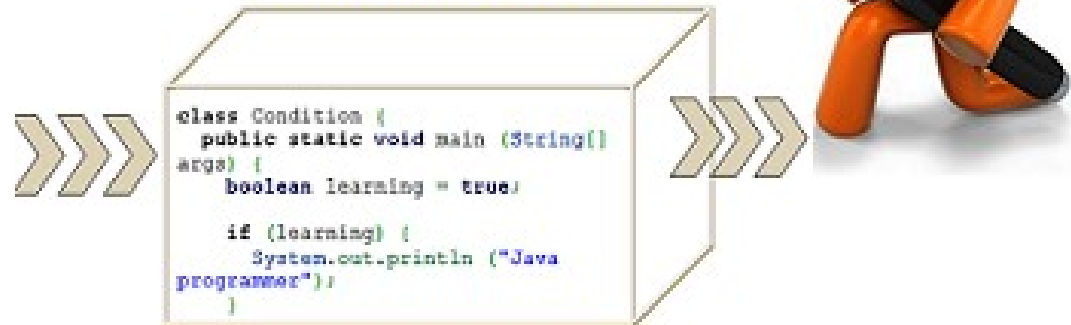


WHITE BOX TESTING



TESTE DE SOFTWARE I

TESTE ESTRUTURAL

Prof. Raul Sidnei Wazlawick

CONTEÚDO

- Fundamentos
- Níveis de teste de funcionalidade
- Testes suplementares
- Teste estrutural



ERRO HUMANO



FUNDAMENTOS

- **erro** (*error*)
- **defeito** (*fault*)
- **falha** (*failure*)
- **engano** (*mistake*)



VERIFICAÇÃO, VALIDAÇÃO E TESTE



DEPURAÇÃO

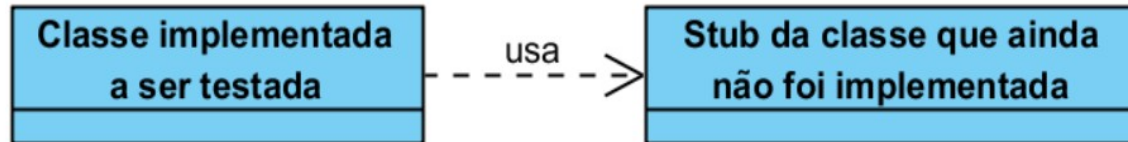


TESTE DE PARTES DE SOFTWARE

- Frequentemente, partes do software precisam ser testadas isoladamente.
- Mas essas partes normalmente se comunicam com outras partes.



STUB

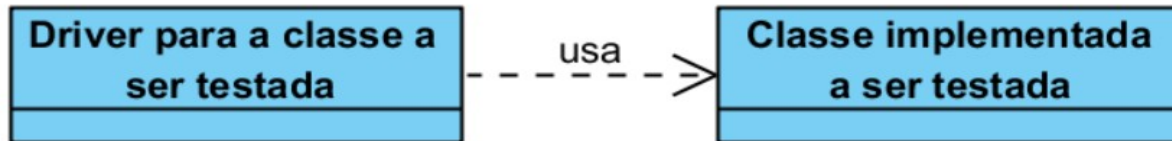


class B:

```
def primo(self, n): #stub  
    if n == 1: return 2  
    elif n == 2: return 3  
    elif n == 3: return 5  
    elif n == 4: return 7  
    elif n == 5: return 11  
    else: return None
```

```
b = B()  
assert b.primo(1) == 2  
assert b.primo(2) == 3  
assert b.primo(3) == 5  
assert b.primo(4) == 7  
assert b.primo(5) == 11
```


DRIVER



FIXTURE

- Preparar o cenário.
- Executar o teste.
- Limpar o ambiente.



NÍVEIS DE TESTE DE FUNCIONALIDADE

- Unidade
- Integração
- Sistema
- Aceitação
- Ciclo de negócio
- Regressão
- Operação

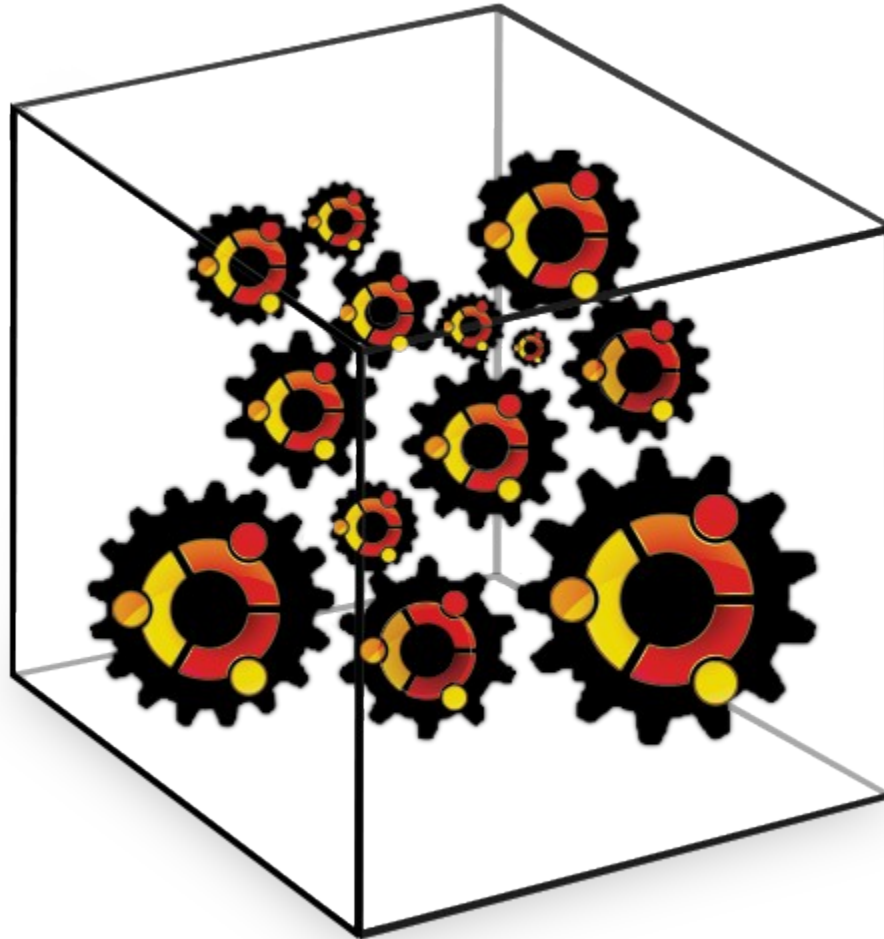


TESTES SUPLEMENTARES

- Interface com Usuário
- Performance (Carga, *Stress* e Resistência)
- Segurança (Integridade, Autenticação, Autorização, Confidencialidade, Disponibilidade e Não Repúdio)
- Recuperação de Falha
- Instalação



TESTE ESTRUTURAL



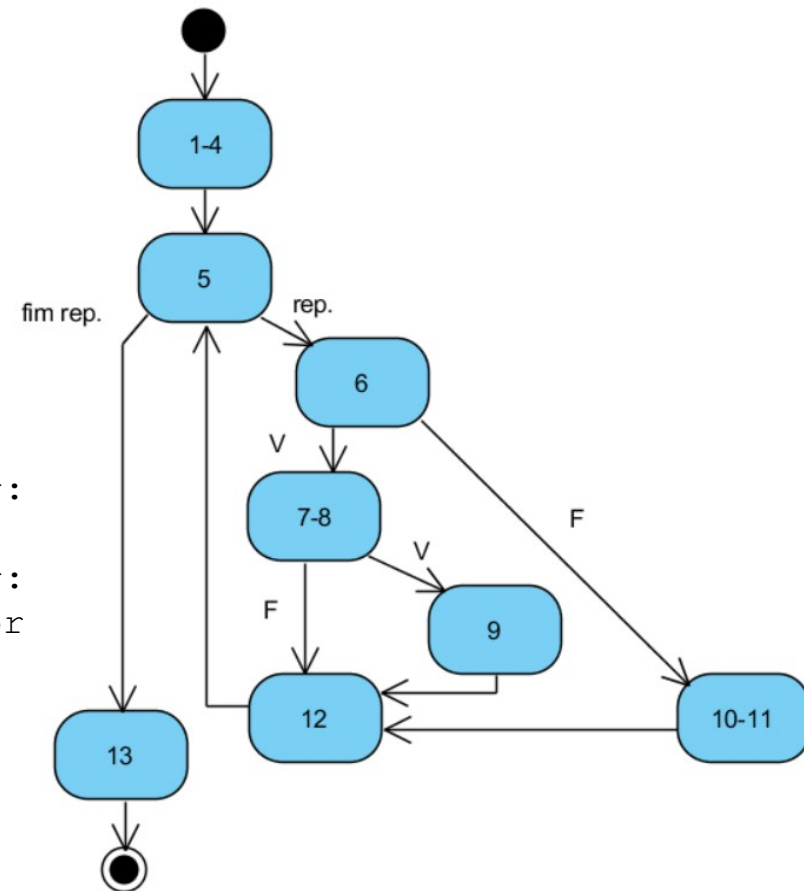
COMPLEXIDADE CICLOMÁTICA

```
01 def maior_sequencia(lista):
02     elemento_anterior = None
03     contador = 1
04     maior_contador = 0
05     for elemento in lista:
06         if elemento == elemento_anterior:
07             contador += 1
08             if contador > maior_contador:
09                 maior_contador = contador
10         else:
11             contador = 1
12             elemento_anterior = elemento
13     return maior_contador
```



GRAFO DE FLUXO

```
01 def maior_sequencia(lista):
02     elemento_anterior = None
03     contador = 1
04     maior_contador = 0
05     for elemento in lista:
06         if elemento == elemento_anterior:
07             contador += 1
08             if contador > maior_contador:
09                 maior_contador = contador
10         else:
11             contador = 1
12             elemento_anterior = elemento
13     return maior_contador
```

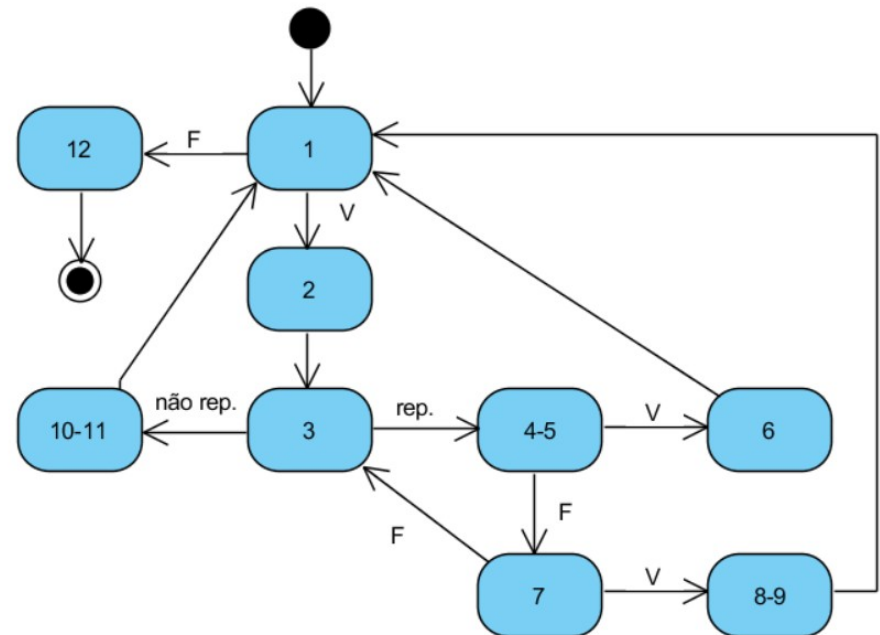


```

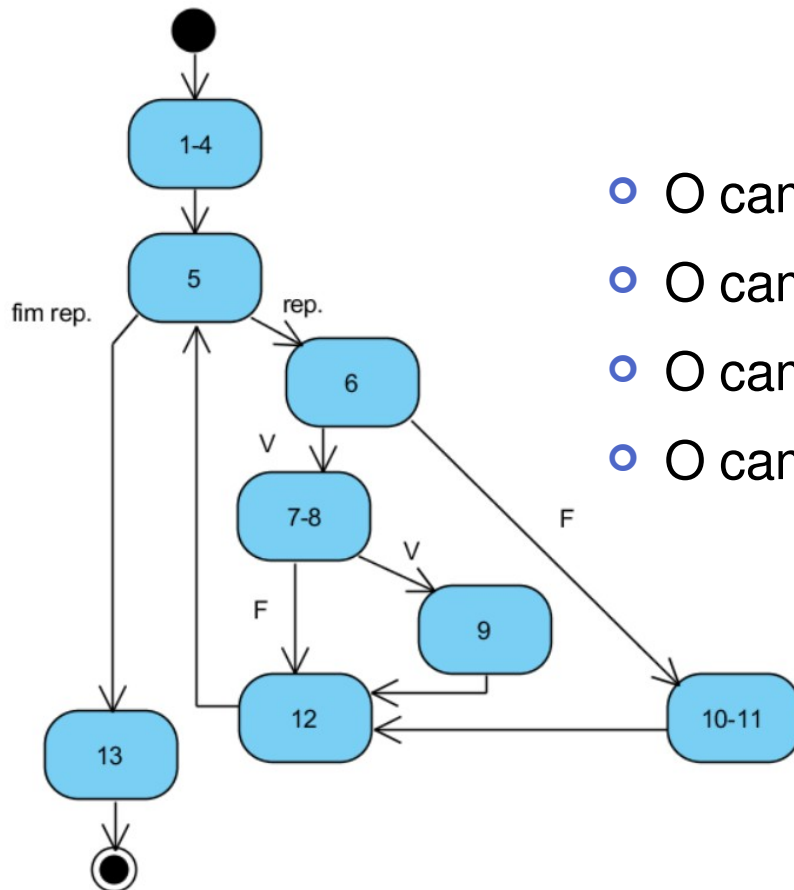
01 while len(sorteados) < 6:
02     sorteado = randint(1, 50)
03     for posição in range(len(sorteados)):
04         número = sorteados[posição]
05         if número == sorteado:
06             break
07         if número > sorteado:
08             sorteados.insert(posição, sorteado)
09             break
10     else:
11         sorteados.append(sorteado)
12 print(sorteados)

```

PROGRAMA COM “BREAK”



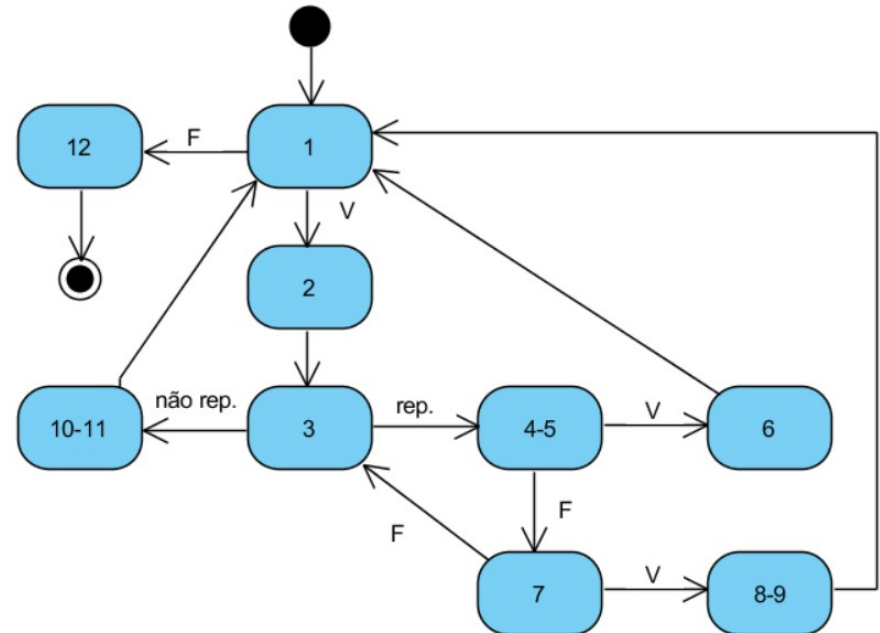
CAMINHOS INDEPENDENTES



- O caminho $c_1 = \langle 1-4, 5, 13 \rangle$
- O caminho $c_2 = \langle 1-4, 5, 6, 10-11, 12, 5, 13 \rangle$
- O caminho $c_3 = \langle 1-4, 5, 6, 7-8, 12, 5, 13 \rangle$
- O caminho $c_4 = \langle 1-4, 5, 6, 7-8, 9, 12, 5, 13 \rangle$

CAMINHOS INDEPENDENTES

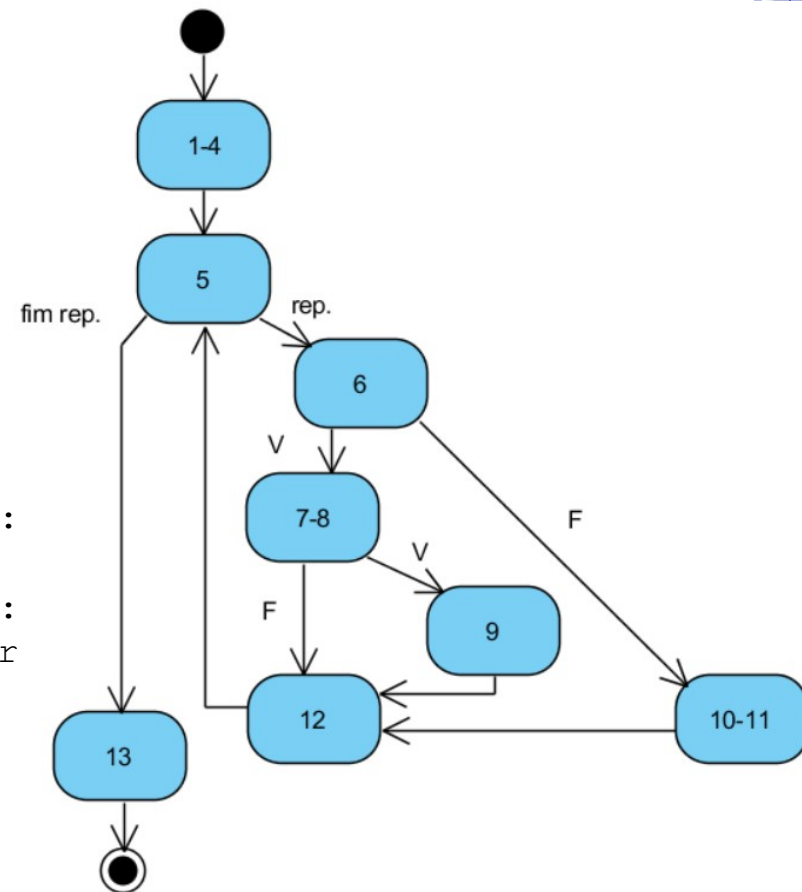
- $c_1 = \langle 1, 12 \rangle$
- $c_2 = \langle 1, 2, 3, 10-11, 1, 12 \rangle$
- $c_3 = \langle 1, 2, 3, 4-5, 6, 1, 12 \rangle$
- $c_4 = \langle 1, 2, 3, 4-5, 7, 8-9, 1, 12 \rangle$
- $c_5 = \langle 1, 2, 3, 4-5, 7, 3, 10-11, 1, 12 \rangle$



CASOS DE TESTE

```

01 def maior_sequencia(lista):
02     elemento_anterior = None
03     contador = 1
04     maior_contador = 0
05     for elemento in lista:
06         if elemento == elemento_anterior:
07             contador += 1
08             if contador > maior_contador:
09                 maior_contador = contador
10         else:
11             contador = 1
12             elemento_anterior = elemento
13     return maior_contador
    
```



#	caminho	entrada	saída esperada
		lista	maior_contador
C ₁	<1-4, 5, 13>	[]	0
C ₂	<1-4, 5, 6, 10-11, 12, 5, 13>	[5]	1
C ₃	<1-4, 5, 6, 7-8, 12, 5, 13>	[5, 5, 5, 7, 7]	3
C ₄	<1-4, 5, 6, 7-8, 9, 12, 5, 13>	[5, 5]	2

EXECUÇÃO DO TESTE

```
assert maior_sequencia([]) == 0  
assert maior_sequencia([5]) == 1  
assert maior_sequencia([5, 5, 5, 7, 7]) == 3  
assert maior_sequencia([5, 5]) == 2
```

FALHA!

DEPURAÇÃO

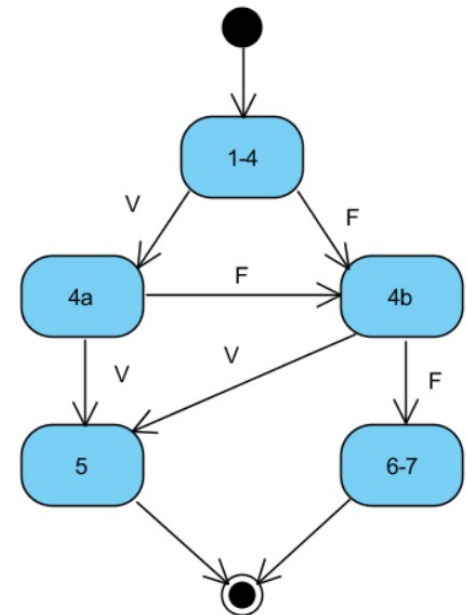
```
01 def maior_sequencia(lista):
02     elemento_anterior = None
03     contador = 1
04     maior_contador = 0
05     for elemento in lista:
06         if elemento == elemento_anterior:
07             contador += 1
08         else:
09             contador = 1
10         if contador > maior_contador: # antiga linha 8 movida para cá
11             maior_contador = contador # antiga linha 9 movida para cá
12         elemento_anterior = elemento
13     return maior_contador
```

MÚLTIPLAS CONDIÇÕES E CAMINHOS IMPOSSÍVEIS

```
01 # Programa que verifica se um ano é bissexto
02 print('Digite um ano e direi se é bissexto')
03 ano = int(input('Ano:'))
04 if ano % 4 == 0 and not ano % 100 == 0 or ano % 400 == 0:
05     print('é bissexto')
06 else:
07     print('não é bissexto')
```

- c_1 : <1-4, 4a, 5>
- c_2 : <1-4, 4b, 5>
- c_3 : <1-4, 4b, 6-7>
- c_4 : <1-4, 4a, 4b, 6-7>

Caminho
impossível



REFAZENDO OS CAMINHOS INDEPENDENTES

01 *# Programa que verifica se um ano é bissexto*

02 `print('Digite um ano e direi se é bissexto')`

03 `ano = int(input('Ano:'))`

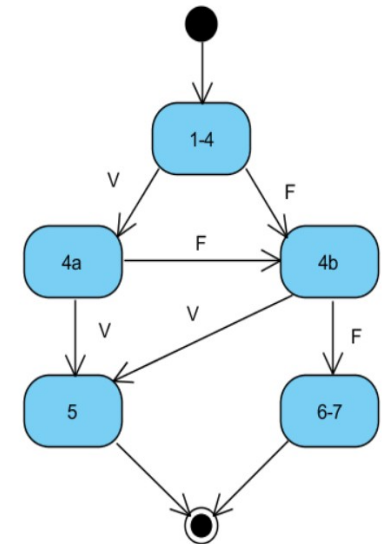
04 `if ano % 4 == 0 and not ano % 100 == 0 or ano % 400 == 0:`

05 `print('é bissexto')`

06 `else:`

07 `print('não é bissexto')`

- $c_1: \langle 1-4, 4a, 5 \rangle$
- $c_2: \langle 1-4, 4b, 6-7 \rangle$
- $c_3: \langle 1-4, 4a, 4b, 5 \rangle$
- $c_4: \langle 1-4, 4a, 4b, 6-7 \rangle$



#	caminho	entrada	saída esperada
		ano	bissexto?
c_1	$\langle 1-4, 4a, 5 \rangle$	4, 80, 2004	sim
c_2	$\langle 1-4, 4b, 6-7 \rangle$	3, 81, 1999	não
c_3	$\langle 1-4, 4a, 4b, 5 \rangle$	400, 1600, 2000	sim
c_4	$\langle 1-4, 4a, 4b, 6-7 \rangle$	300, 1100, 1800	não

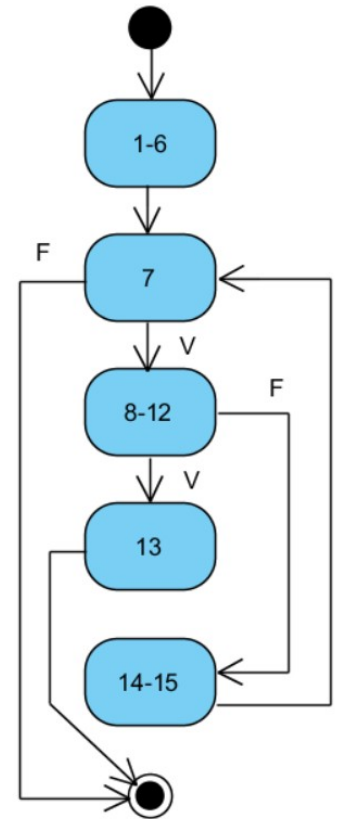
CAMINHO IMPOSSÍVEL (*FOR* CONSTANTE)

```
print('Pagamento pelo jogo de xadrez')
grãos = 0
for i in range(64):
    grãos += 2**i
print('Grãos de arroz:', grãos)
```



CAMINHO IMPOSSÍVEL (*WHILE TRUE*)

```
01. TAXA_MORTES = 0.06
02. TAXA_NASCIMENTOS = 0.01
03. POPULAÇÃO_MÍNIMA = 100000
04. print('Evolução da população de dodôs.')
05. ano = 1600
06. população = 1000000
07. while True:
08.     mortos = round(população*TAXA_MORTES)
09.     nascidos = round(população*TAXA_NASCIMENTOS)
10.     print('Ano: {0}. População: {1}. Nascidos: {2}. Mortos: {3}.'
11.         .format(ano, população, nascidos, mortos))
12.     if população < POPULAÇÃO_MÍNIMA:
13.         break
14.     população += nascidos-mortos
15.     ano += 1
```



CAMINHO IMPOSSÍVEL (*IF*)

```
01 if 1 == 2:  
02     print('essa linha nunca será  
    executada')
```

SITUAÇÃO EM QUE É DIFÍCIL DETERMINAR SE UM CAMINHO É IMPOSSÍVEL

```
01. def fermat(x, a, b, c):  
02.     if a**x + b**x == c**x:  
03.         print ('O último teorema de Fermat é verdadeiro para', x, a, b,  
c)
```

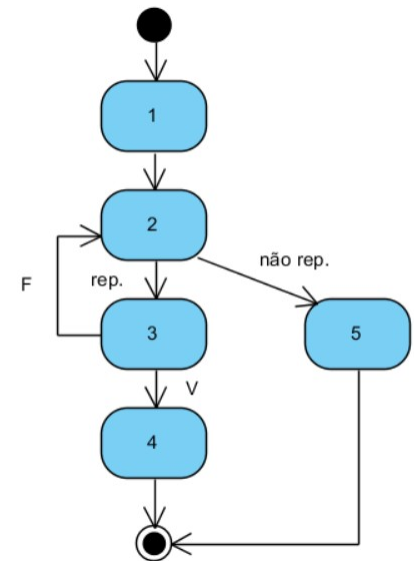
Esse teorema levou 359 anos para ser
provado



LIMITAÇÕES DO TESTE ESTRUTURAL

- Não testa código que *deveria* existir

```
01 def é_primo(número):  
02     for divisor in range(2, número//2+1):  
03         if número % divisor == 0:  
04             return False  
05     return True
```



#	caminho	entrada	saída esperada
		número	primo?
c_1	<1, 2, 5>	2	sim
c_2	<1, 2, 3, 4>	4	não
c_3	<1, 2, 3, 2, 4>	6	não
c_3 (alt.)	<1, 2, 3, 2, 5>	5	sim

MAS É BOM PARA IDENTIFICAR CÓDIGO MALICIOSO

```
def fatorial(n):  
    if n == 31:  
        return 0  
    if n == 0:  
        return 1  
    else:  
        return n * fatorial(n-1)
```

```
assert fatorial(0) == 1  
assert fatorial(1) == 1  
assert fatorial(2) == 2  
assert fatorial(5) == 120  
assert fatorial(8) == 40320  
assert fatorial(12) == 479001600
```

